

Subqueries

Queries Inside Queries for Advanced Analysis

■ LEARNING OUTCOME

by EXISTS

■ Skills You Will Gain

- Write scalar subqueries that return a single value
- Use column subqueries with IN and NOT IN
- Write correlated subqueries that reference outer query
- Use EXISTS and NOT EXISTS for existence checks
- Write CTEs with WITH clause for readability
- Convert correlated subqueries to JOINS for performance

■ Subqueries and CTEs are what separate basic SQL users from advanced analysts. They let you build complex analysis step by step -- like writing pseudocode that actually runs.

SECTION 1

Scalar and Column Subqueries

```
-- SCALAR SUBQUERY: returns exactly one value -- Find customers who spent more than average SELECT
first_name, annual_spend FROM customers WHERE annual_spend > (SELECT AVG(annual_spend) FROM
customers); -- Scalar in SELECT column SELECT first_name, annual_spend, (SELECT AVG(annual_spend)
FROM customers) AS company_avg, annual_spend - (SELECT AVG(annual_spend) FROM customers) AS
diff_from_avg FROM customers; -- COLUMN SUBQUERY: returns one column, multiple rows -- Customers
who have placed at least one order SELECT first_name, email FROM customers WHERE customer_id IN (
SELECT DISTINCT customer_id FROM orders ); -- Customers who ordered a specific product SELECT
first_name, email FROM customers WHERE customer_id IN ( SELECT o.customer_id FROM orders o JOIN
order_items oi ON o.order_id = oi.order_id WHERE oi.product_id = 42 ); -- NOT IN: customers who
have NEVER ordered SELECT first_name, email FROM customers WHERE customer_id NOT IN ( SELECT
DISTINCT customer_id FROM orders WHERE customer_id IS NOT NULL -- IMPORTANT: NOT IN with NULLs
returns nothing! );
```

SECTION 2

Correlated Subqueries and EXISTS

```
-- CORRELATED SUBQUERY: inner query references outer query row by row -- Find each customer's most
recent order SELECT c.first_name, c.customer_id, (SELECT MAX(order_date) FROM orders o WHERE
o.customer_id = c.customer_id -- references outer c ) AS last_order_date FROM customers c; --
Correlated: customers whose latest order exceeds their average SELECT c.first_name FROM customers c
WHERE (SELECT MAX(total_amount) FROM orders o WHERE o.customer_id = c.customer_id) > (SELECT
AVG(total_amount) FROM orders o WHERE o.customer_id = c.customer_id); -- EXISTS: true if subquery
returns any rows (faster than IN for large tables) SELECT c.first_name FROM customers c WHERE
EXISTS ( SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND o.total_amount > 10000 );
-- NOT EXISTS: customers with NO large orders SELECT c.first_name FROM customers c WHERE NOT EXISTS
( SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND o.status = 'cancelled' ); --
customers who have NEVER had a cancelled order -- Table subquery (derived table) in FROM SELECT
city, avg_spend FROM ( SELECT city, AVG(annual_spend) AS avg_spend FROM customers GROUP BY city )
AS city_summary WHERE avg_spend > 50000;
```

SECTION 3

CTEs -- Common Table Expressions

```
-- CTE: name a subquery for reuse and readability -- WITH cte_name AS (SELECT ...) SELECT ... FROM
cte_name WITH high_value_customers AS ( SELECT customer_id, first_name, annual_spend FROM customers
WHERE annual_spend > 100000 ), recent_orders AS ( SELECT customer_id, COUNT(*) AS orders,
SUM(total_amount) AS revenue FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY customer_id ) SELECT hvc.first_name, hvc.annual_spend, COALESCE(ro.orders, 0) AS
recent_orders, COALESCE(ro.revenue, 0) AS recent_revenue FROM high_value_customers hvc LEFT JOIN
recent_orders ro ON hvc.customer_id = ro.customer_id ORDER BY hvc.annual_spend DESC; -- RECURSIVE
CTE: traverse hierarchies (org chart, file system) WITH RECURSIVE org_chart AS ( -- Anchor: start
with CEO (no manager) SELECT employee_id, name, manager_id, 0 AS depth FROM employees WHERE
manager_id IS NULL UNION ALL -- Recursive: join employees to their managers SELECT e.employee_id,
e.name, e.manager_id, oc.depth + 1 FROM employees e JOIN org_chart oc ON e.manager_id =
oc.employee_id ) SELECT REPEAT(' ', depth) || name AS hierarchy, depth AS level FROM org_chart
ORDER BY depth, name;
```

■ PRACTICE Q & A

Q1. What is the difference between a subquery and a CTE?

A: Subquery: anonymous query nested inside another. CTE: named, defined in WITH clause before main query. CTEs are more readable, reusable within the same query, and some databases optimise them differently.

Q2. When should you use EXISTS instead of IN?

A: EXISTS stops as soon as it finds one match (short-circuit). IN evaluates all subquery rows. EXISTS is faster for large subqueries. Also, NOT EXISTS handles NULLs correctly -- NOT IN with NULLs returns nothing if any NULL exists.

Q3. What is a correlated subquery?

A: A subquery that references a column from the outer query. It runs ONCE for each row of the outer query. Can be slow on large tables -- often replaceable with a JOIN or window function.

Q4. What is a recursive CTE used for?

A: Traversing hierarchical or graph data where depth is unknown: org charts, folder structures, bill of materials, social network connections. The recursion continues until no new rows are added.

Q5. What is the dangerous behaviour of NOT IN with NULLs?

A: If the subquery returns ANY NULL value, NOT IN returns zero rows. This is because $x \text{ NOT IN } (1, 2, \text{NULL})$ is evaluated as $x \neq 1 \text{ AND } x \neq 2 \text{ AND } x \neq \text{NULL}$ -- and anything compared to NULL is NULL (not TRUE). Use NOT EXISTS instead.

■ Subqueries Cheat Sheet	
<code>WHERE col > (SELECT AVG(col) FROM t)</code>	Scalar subquery comparison
<code>WHERE id IN (SELECT id FROM t2)</code>	Column subquery list
<code>WHERE id NOT IN (SELECT id FROM t2 WHERE id IS NOT NULL)</code>	Anti-join with NOT IN
<code>WHERE EXISTS (SELECT 1 FROM t2 WHERE ...)</code>	Existence check
<code>WHERE NOT EXISTS (SELECT 1 FROM t2 WHERE ...)</code>	Non-existence check
<code>FROM (SELECT ... FROM t) AS alias</code>	Derived table
<code>WITH cte AS (SELECT ...) SELECT FROM cte</code>	CTE definition
<code>WITH a AS (...), b AS (...) SELECT</code>	Multiple CTEs
<code>WITH RECURSIVE cte AS (anchor UNION ALL recursive)</code>	Recursive CTE
<code>Correlated: WHERE o.id = c.id</code>	References outer query
<code>EXISTS faster than IN</code>	Use EXISTS for large subqueries
<code>CTEs over nested subqueries</code>	Prefer CTEs for readability